

UNIVERSIDAD
NACIONAL
DE COLOMBIA

Proyecto

Sistema de Gestión de Inventarios basado en Microservicios para

FerreAndina S.A.S

Entrega 2 ~ Arquitectura de Microservicios y Modelos Relacionales

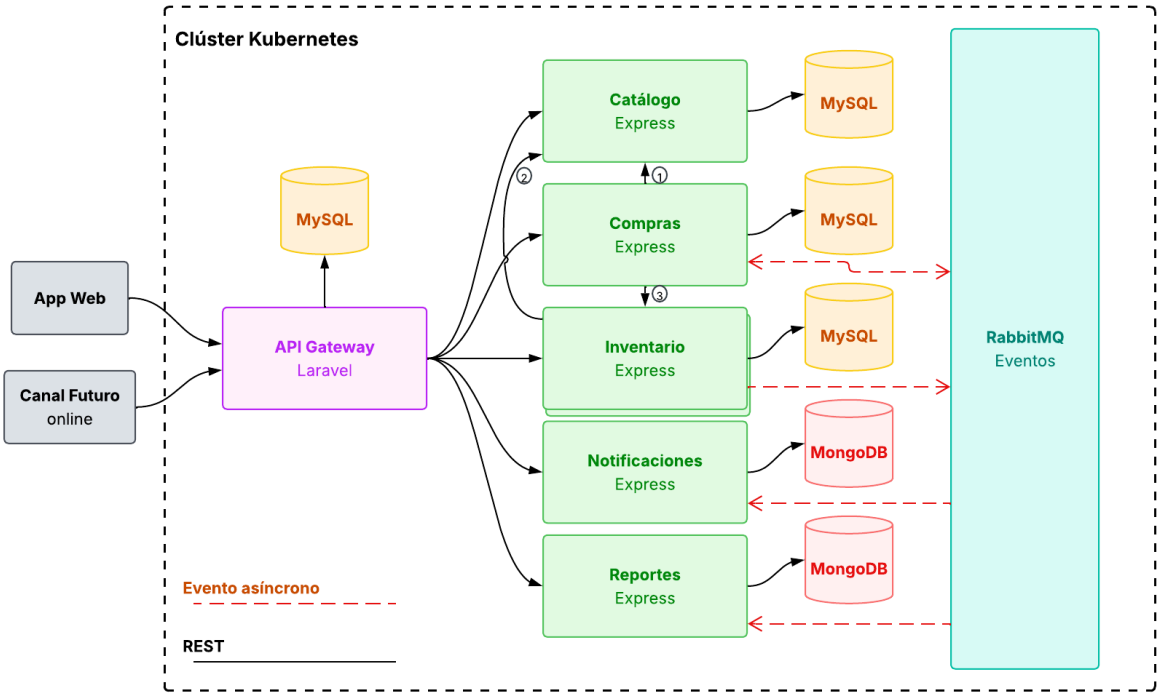
Luis Felipe Giraldo Ortega
Steven Lopez Parra
Jhoan Sebastian Martinez Martinez

Universidad Nacional de Colombia
Departamento de Informática y Computación
Ingeniería de Software II
Manizales, Colombia

2026-2

1. Diagrama de arquitectura de microservicios

El sistema está formado por un **API Gateway propio** y **cinco microservicios de negocio** (Catálogo, Compras, Inventario, Notificaciones y Reportes), desplegados como contenedores en un clúster Kubernetes. Los clientes (la aplicación web de los 15 usuarios internos (aprox.) y, a futuro, un canal de venta en línea) solo hablan con el Gateway. Entre servicios hay dos formas de comunicación: **REST**, cuando quien llama necesita la respuesta para continuar, y **eventos asíncronos por RabbitMQ**, cuando solo se avisa que algo ocurrió.



El cliente web y el futuro canal en línea (RF15) solo hablan con el API Gateway (Laravel), que valida el JWT, revisa el rol y enruta la petición por REST al servicio que corresponde (RNF09). Catálogo, Compras e Inventario usan MySQL; Notificaciones y Reportes, MongoDB. Cada servicio tiene su propia base de datos (RNF02) y todo corre dentro del clúster de Kubernetes.

Entre servicios usamos REST cuando la operación necesita la respuesta en el momento: Compras valida un producto en Catálogo, y al recibir una orden registra la entrada de mercancía en Inventario. Lo demás viaja como eventos por RabbitMQ (sección 1.3).

1.1 Comunicación entre microservicios

Criterio: ¿REST o evento?

- **REST** cuando quien llama necesita la respuesta para seguir (validar un producto, registrar una entrada).
- **Evento** cuando solo avisa que algo pasó y no espera respuesta (stock bajo, movimiento registrado). Si el receptor está caído, quien publica no se detiene.
- Las peticiones **externas** siempre pasan por el Gateway. Las llamadas **entre servicios** van directo, por el nombre del servicio (por ejemplo catalogo-svc) que resuelve Kubernetes. Los servicios no se exponen fuera del clúster; solo el Gateway.

1.2 Relaciones directas entre servicios (por REST)

#	Origen → Destino	Para qué	Si el destino falla
①	**Compras → Catálogo**	Al crear una orden de compra: validar que el producto existe y obtener su proveedor asociado (también al generar la orden sugerida del RF16).	No se puede crear la orden (error 503). Consultar o recibir órdenes ya creadas no depende de Catálogo, porque la orden guarda una copia del nombre del producto.
②	**Inventario → Catálogo**	Al registrar un movimiento o devolución: validar el producto y traer su código, nombre, categoría y costo de referencia (datos que viajan en el evento hacia Reportes).	El movimiento no se registra (503). Mejora posible: caché corta de productos.
③	**Compras → Inventario**	Al recibir mercancía: registrar la ENTRADA en la bodega indicada, con referencia a la orden.	La recepción no se confirma y se reintenta; la referencia de la recepción evita duplicar la entrada.

Si además Inventario llamara a Compras en línea habría un ciclo síncrono, y la caída de uno detendría al otro. Esa dirección va por evento (inventario.reabastecimiento.requerido).

1.3 Eventos asíncronos con RabbitMQ

Cómo funciona.

1. El servicio donde ocurre el hecho (**productor**) publica un mensaje en un *exchange* llamado ferreandina.events, con una clave de ruteo como inventario.stock.bajo.
2. El exchange, de tipo *topic*, reparte el mensaje a las **colas** enlazadas con esa clave.
3. Cada servicio consumidor tiene su propia cola durable (notificaciones.q, reportes.q, compras.q); allí el mensaje espera hasta que el servicio lo lea.
4. El consumidor procesa el mensaje y confirma (*ack*). Si falla, el mensaje no se pierde y se reintenta.
5. El productor nunca sabe quién consume ni espera respuesta; por eso una caída de Notificaciones o Reportes no frena las bodegas.

Evento	Publica	Consume	Cuándo se publica	Para qué
inventario.movimiento.registrado	Inventario	Reportes	Cada movimiento registrado	Actualizar existencias, rotación e indicadores (resumen del movimiento) (RF11, RF14)
inventario.stock.bajo	Inventario	Notificaciones	Un movimiento deja el saldo bajo el mínimo	Alertar que un producto bajó de su mínimo en una bodega (y envía el correo), con la sugerencia de traslado si la hay (RF09, RF16)
inventario.reabastecimiento.requerido	Inventario	Compras	Stock bajo y ninguna otra bodega tiene excedente (RF16)	Crear una orden de compra en borrador con el proveedor del producto, cuando ninguna sede tiene excedente (RF16)
compras.orden.sugerida	Compras	Notificaciones	Se crea la orden en borrador	Avisar al responsable que hay una orden por revisar y aprobar (RF16)
catalogo.producto.actualizado	Catálogo	Reportes	Se actualice un producto	Mantener la copia de nombres y costos de los productos para valorizar el inventario (RF11)

Contenido mínimo de los eventos. Todos llevan eventoId único, tipo y fecha. Además: stock.bajo incluye producto, bodega, stock actual y mínimo, y la sugerencia de traslado si existe; reabastecimiento.requerido, la sugerencia, el producto, la bodega destino y la cantidad; orden.sugerida, la orden, el producto, el proveedor y la cantidad; movimiento.registrado, el movimiento, los datos del producto (código, nombre, categoría, costo de referencia) y una lista afectaciones con el saldo y el mínimo de cada bodega afectada (una, o dos si es un traslado).

Flujo completo del RF16.

1. Un operador registra una salida y el saldo queda bajo el mínimo.
2. Inventario guarda el movimiento y publica movimiento.registrado; Reportes actualiza sus existencias.

3. Inventario revisa las demás bodegas. Si alguna tiene excedente, crea una sugerencia de **traslado** y publica stock.bajo con ella; Notificaciones avisa y un administrador la acepta o la rechaza.
4. Si ninguna tiene excedente, publica reabastecimiento.requerido; Compras consulta el proveedor a Catálogo (①) y crea una orden en **BORRADOR**.
5. Compras publica orden.sugerida; Notificaciones avisa al responsable. La orden nunca se envía sola: **la decisión es del usuario.**

Garantías y duplicados. RabbitMQ entrega **«al menos una vez»**, así que un evento puede llegar repetido. Cada consumidor lo tolera con un índice único (sección 5): si el mismo evento llega dos veces, la segunda inserción se rechaza y se ignora.

2. Responsabilidades de cada componente

Componente	Responsabilidad	RF	Base de datos
API Gateway (Laravel)	Único punto de entrada: enruta, valida el JWT y aplica roles. Incluye usuarios, roles, registro, inicio y cierre de sesión y recuperación de contraseña.	RF07, RF08	MySQL
Catálogo (Express)	Productos, categorías y subcategorías, búsqueda y filtros, costo de referencia y asociación producto–proveedor.	RF01, RF02, RF12	MySQL
Compras (Express)	Proveedores, órdenes de compra, recepción de mercancía y órdenes sugeridas en borrador.	RF03, RF06, RF16	MySQL
Inventario (Express)	Bodegas, saldos por bodega, movimientos y kardex, devoluciones, sugerencias de traslado y consulta de stock para canales externos.	RF04, RF05, RF10, RF13, RF15, RF16	MySQL
Notificaciones (Express)	Alertas de stock bajo y de reabastecimiento, en interfaz y por correo.	RF09, RF16	MongoDB
Reportes (Express)	Existencias, valorización, rotación y panel de indicadores, sobre una copia de lectura.	RF11, RF14	MongoDB
RabbitMQ	Broker de eventos entre servicios.	RNF 01	—

Por qué Inventario se destaca. Concentra la consulta de stock, la operación más frecuente del negocio, y es el servicio que se escala con réplicas (RNF05). Por eso también atiende la consulta de stock que usará el canal externo (RF15), siempre a través del Gateway.

3. Modelo de datos por microservicio (Diagrama entidad-relación)

Cada servicio tiene su propia base de datos y ninguno lee la de otro (RNF02). Entre servicios no hay llaves foráneas: se guarda el id de la otra entidad como referencia lógica (sección 5.7) y, si hace falta, se valida por REST. Convención: PK clave primaria, FK clave foránea dentro del mismo servicio, UQ valor único.

Convenciones. Tablas y campos van en español, en minúscula y con guión bajo; la llave primaria se llama id (en MongoDB, _id). Los tipos son los del motor de cada servicio: int, varchar, text, decimal, enum, bool, date y datetime en MySQL; ObjectId, string, number, boolean, date, array y Object en MongoDB. Un campo que apunta a otro servicio (por ejemplo producto_id en Inventario) es una referencia lógica: guarda el número, sin llave foránea, y su existencia se valida por REST (RNF02). La marca (FK) solo aparece cuando la tabla referenciada está en la misma base de datos.

Por qué cada servicio tiene base de datos.

Servicio	Motor	Qué guarda	Por qué necesita base propia
Gateway	MySQL	Usuarios, roles y tokens	Es el dueño de las credenciales. Las contraseñas se guardan cifradas (RNF07) y las sesiones cerradas no se pueden perder si el servicio se reinicia.
Catálogo	MySQL	Productos, categorías y su relación con proveedores	Datos maestros con códigos únicos y búsquedas por nombre, código, categoría o proveedor (RF12).
Compras	MySQL	Proveedores, órdenes y recepciones	Una recepción actualiza varias líneas y el estado de la orden en una sola transacción (RF06).
Inventario	MySQL	Bodegas, stock, kardex y sugerencias	El saldo exige consistencia estricta; dos movimientos al mismo tiempo sobre el mismo producto no pueden dejar un saldo equivocado (RF04).
Notificaciones	MongoDB	Alertas, envíos de correo, suscripciones y eventos procesados	Sin base, una alerta o un correo pendiente se pierde si el servicio se reinicia. También guarda qué eventos ya procesó, para no duplicar alertas cuando RabbitMQ reenvía un mensaje.
Reportes	MongoDB	Copia resumida de movimientos, existencias, productos e indicadores	**Ver el párrafo siguiente.**

La pregunta obvia es **¿por qué Reportes no consulta con SQL las tablas de Inventario?**. Hacerlo rompe RNF02: Reportes leería la base de otro servicio, y cualquier cambio en el esquema de Inventario dañaría los reportes. La otra opción, pedirle a Inventario todo el kardex por REST cada vez que alguien abre el dashboard, carga al servicio con más tráfico (RNF05) y deja al sistema sin reportes cuando Inventario está caído. Por eso Reportes mantiene su propia copia, alimentada por los eventos movimiento.registrado y producto.actualizado, con los datos ya agregados por día, bodega y producto; así el dashboard responde en menos de 3 segundos (RF14) sin tocar a Inventario.

Elegimos MongoDB para Reportes y Notificaciones porque guardan documentos que llegan por eventos y se leen casi siempre completos (un indicador diario, una alerta con su referencia), y porque los indicadores pueden crecer (RF14 dice «entre otros») sin migrar tablas. MySQL también serviría; lo que nos da MongoDB aquí es la flexibilidad del esquema, no más rendimiento.

3.1 Gateway

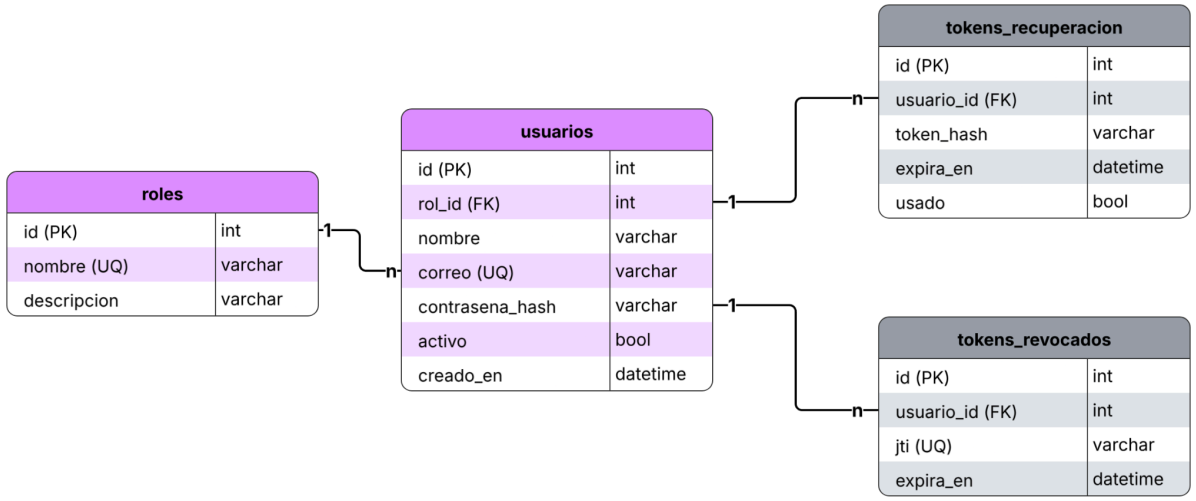


Tabla	Qué guarda	Por qué así	RF
roles	Los roles del sistema: ADMIN, OPERADOR y CANA (a futuro).	nombre es único. Los permisos de cada rol los revisa el Gateway; si más adelante hay más roles, se agrega una tabla de permisos.	RF07
usuarios	Las personas que usan el sistema y su rol.	correo es único porque con él se inicia sesión. La contraseña se guarda cifrada (contrasena_hash), nunca en texto plano. activo permite desactivar a alguien sin borrarlo.	RF07, RF08
tokens_recuperacion	Los enlaces de «olvidé mi contraseña».	Guarda el token cifrado, cuándo vence y si ya se usó, para que cada enlace sirva una sola vez.	RF08

tokens_revocados	Las sesiones cerradas.	Un token JWT sigue siendo válido hasta que vence. Al cerrar sesión se guarda su identificador (jti) y el Gateway rechaza ese token.	RF08

Referencias: Los demás servicios guardan usuario_id como un número, sin llave foránea hacia esta base (RNF02).

3.2 Catálogo

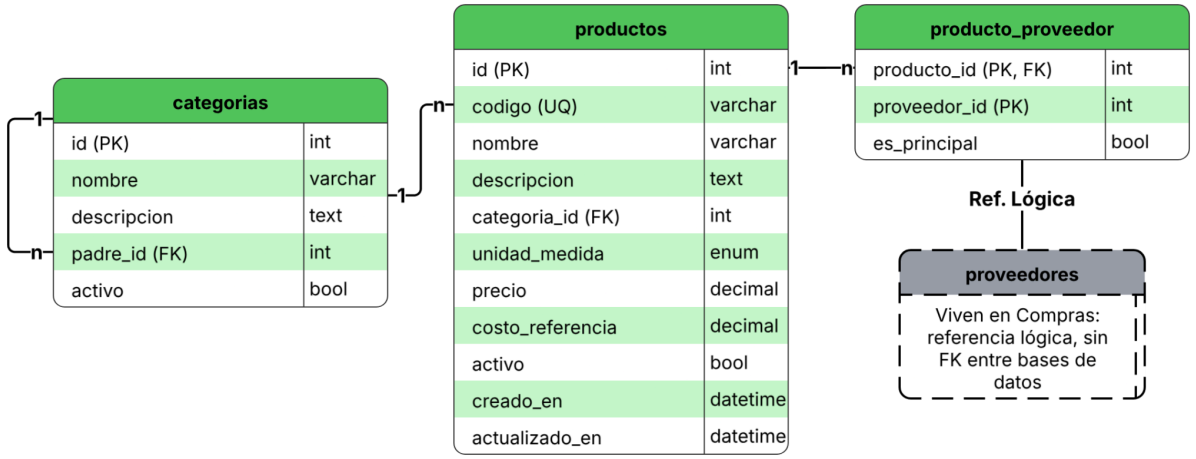


Tabla	Qué guarda	Por qué así	RF
categorias	Categorías y subcategorías de productos.	Si padre_id está vacío es una categoría principal; si tiene valor, es subcategoría de otra. Una sola tabla sirve para cualquier nivel.	RF02
productos	Las cerca de 3.800 referencias.	codigo es único. unidad_medida es una lista fija (UND, KG, M, LT, GAL, CAJA). precio es el de venta y costo_referencia el que se usa para valorizar el inventario. activo sirve para dar de baja sin borrar el historial.	RF01, RF12
producto_proveedor	Qué proveedores surten cada producto.	Un producto puede tener varios proveedores y un proveedor varios productos. es_principal marca a quién se le compra por defecto. proveedor_id apunta a Compras sin llave foránea.	RF03, RF16

Notas: Se indexan código, nombre y categoria_id para que la búsqueda responda en menos de 2 segundos (RF12, RNF06). Cuando cambia el nombre o el costo de un producto, Catálogo avisa a Reportes con el evento producto.actualizado.

3.3 Compras

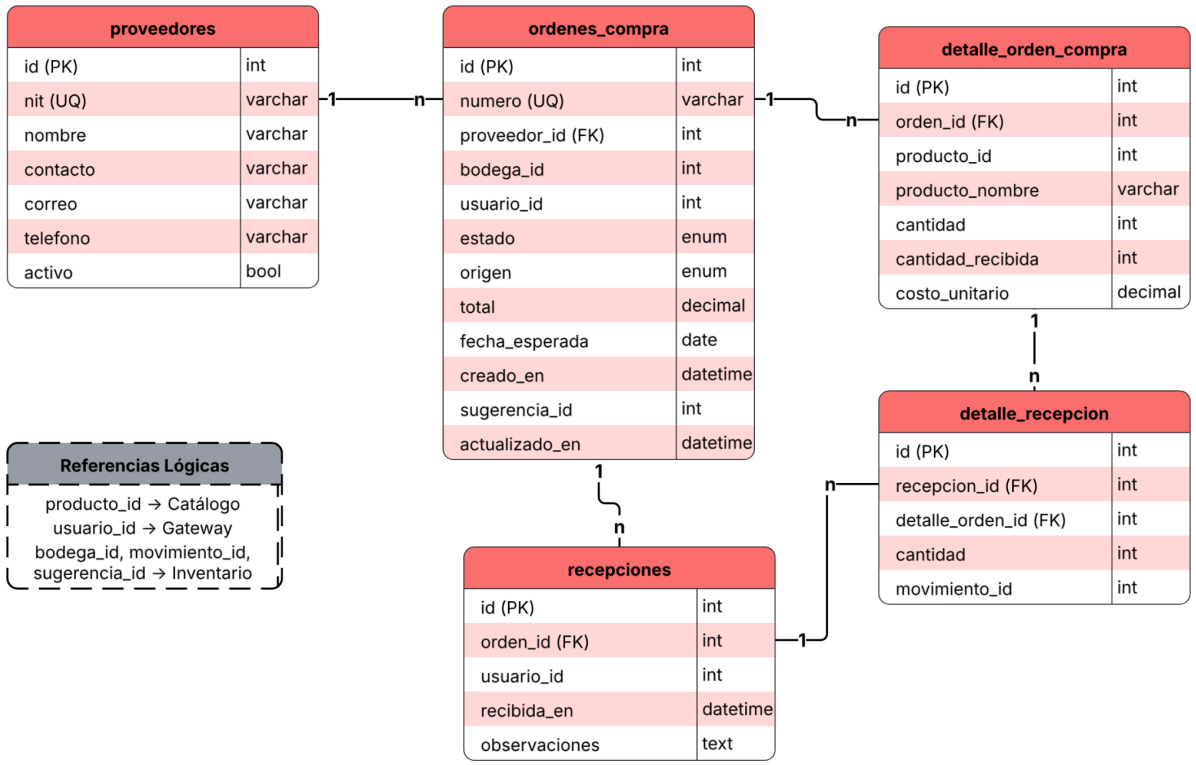


Tabla	Qué guarda	Por qué así	RF
proveedores	Los cerca de 60 proveedores.	nit es único. activo sirve para dar de baja sin perder las órdenes viejas.	RF03
ordenes_compra	Las órdenes de compra.	estado sigue el ciclo BORRADOR → ENVIADA → RECIBIDA_PARCIAL → RECIBIDA, o CANCELADA. origen dice si la orden la creó una persona (MANUAL) o una sugerencia de Inventario (SUGERIDA, con su sugerencia_id). bodega_id es la bodega que recibe.	RF06, RF16
detalle_orden_compra	Los productos de cada orden.	Copia producto_nombre para que la orden se lea igual aunque el producto cambie después. costo_unitario es el precio pactado y cantidad_recibida suma lo que ya llegó.	RF06
recepciones	Cada entrega de mercancía.	Una orden puede llegar en varias entregas, cada una con quién la recibió y cuándo.	RF06
detalle_recepcion	Qué llegó en cada entrega.	Por cada línea recibida, Compras registra la entrada en Inventario y guarda el movimiento_id que le devuelve. Cuando todo llega, la orden pasa a RECIBIDA.	RF04, RF06

movimiento_id guarda la entrada generada en Inventario por la llamada ③.
Referencias: producto_id, usuario_id, bodega_id, movimiento_id y sugerencia_id apuntan a otros servicios, sin llave foránea (RNF02).

3.4 Inventario

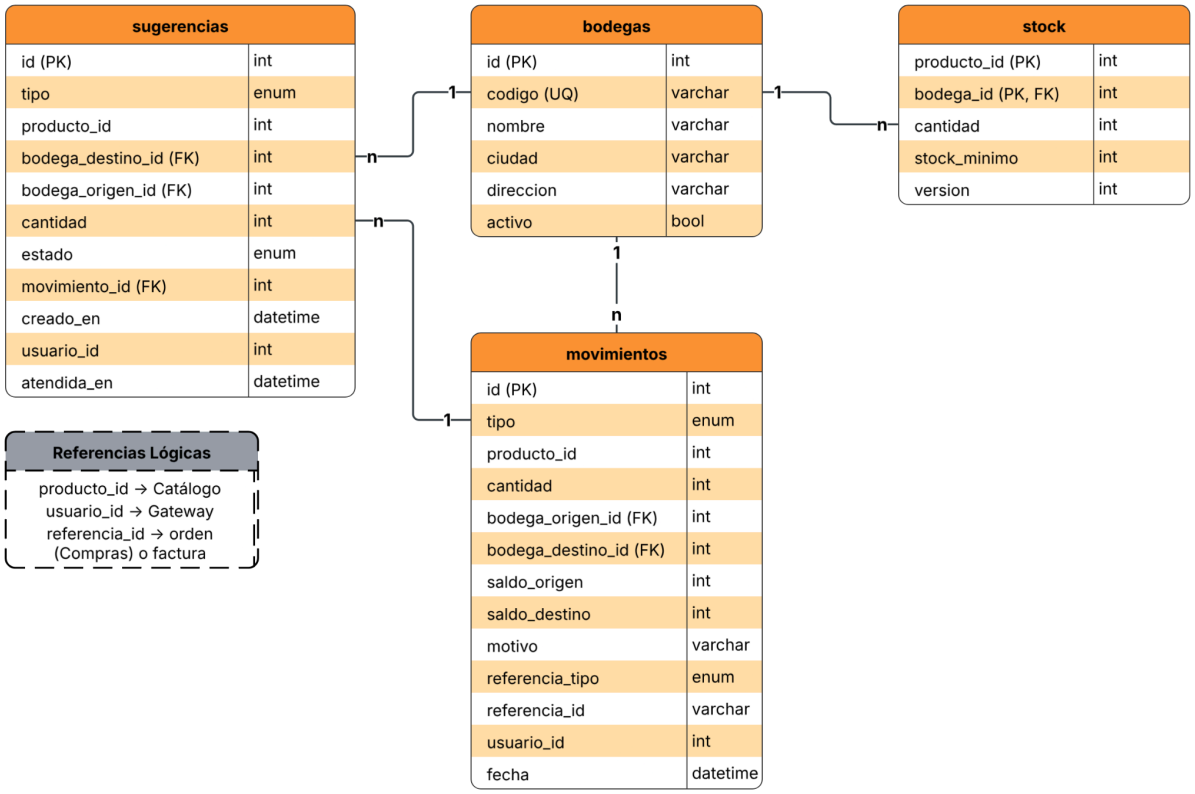


Tabla	Qué guarda	Por qué así	RF
bodegas	Las sedes: Manizales, Pereira y Armenia.	Abrir una sede nueva es agregar una fila. Activa permite dar de baja una sede sin perder su historial de movimientos correspondiente.	RF05
stock	Cuánto hay de cada producto en cada bodega.	La llave es la pareja (producto_id, bodega_id). stock_minimo es el umbral de la alerta. Version evita que dos movimientos registrados al mismo tiempo dejen un saldo equivocado.	RF04, RF05, RF09
movimientos	El kardex: entradas, salidas, ajustes, traslados y devoluciones.	Guarda quién, cuándo, qué y cuánto, y el saldo que quedó. Las devoluciones son un tipo de movimiento, no una tabla aparte, para no repetir datos. referencia_tipo y referencia_id dicen de dónde salió: una orden de compra, una factura o una sugerencia. saldo_origen y saldo_destino guardan el monto restante una vez producido el movimiento, así el kardex se lee sin recalcular toda la historia.	RF04, RF10, RF13
sugerencias	Las sugerencias de reabastecimiento.	Cuando un producto baja del mínimo, se sugiere traerlo de otra sede con excedente (TRASLADO) o comprarlo (COMPRA). El usuario la acepta o la rechaza, y queda quién y cuándo.	RF16

Referencias: producto_id apunta a Catálogo, usuario_id al Gateway y referencia_id a Compras o a un documento externo, sin llave foránea (RNF02).

3.5 Notificaciones

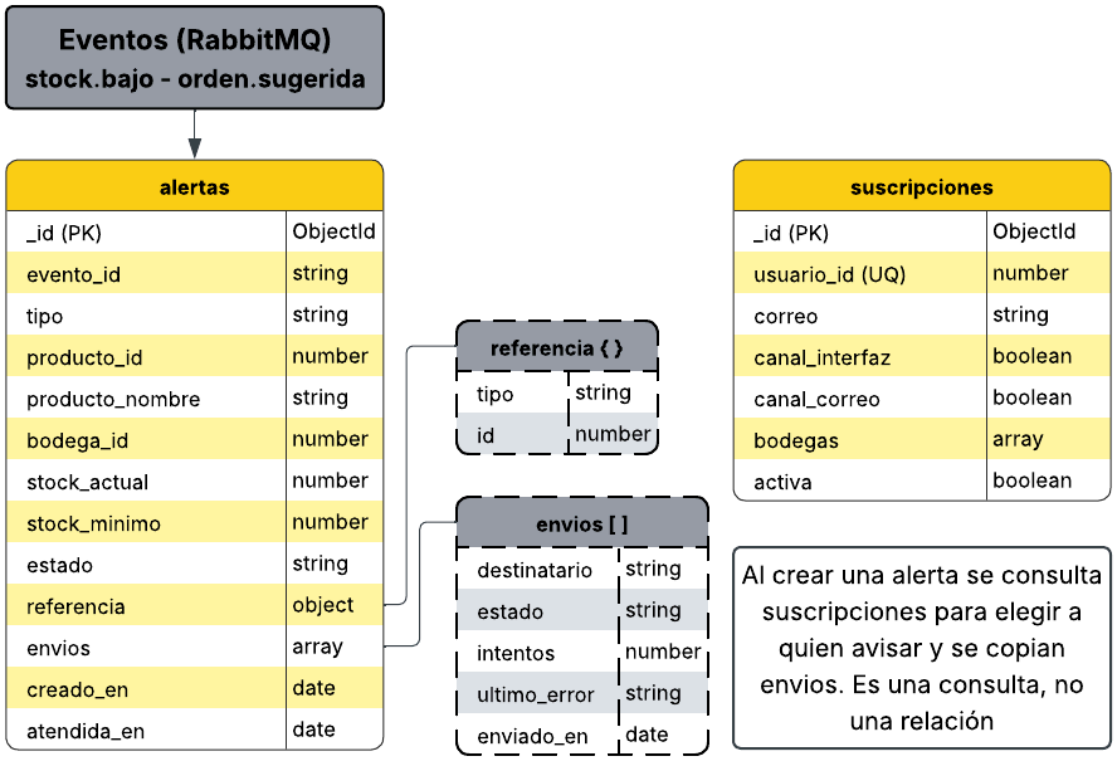
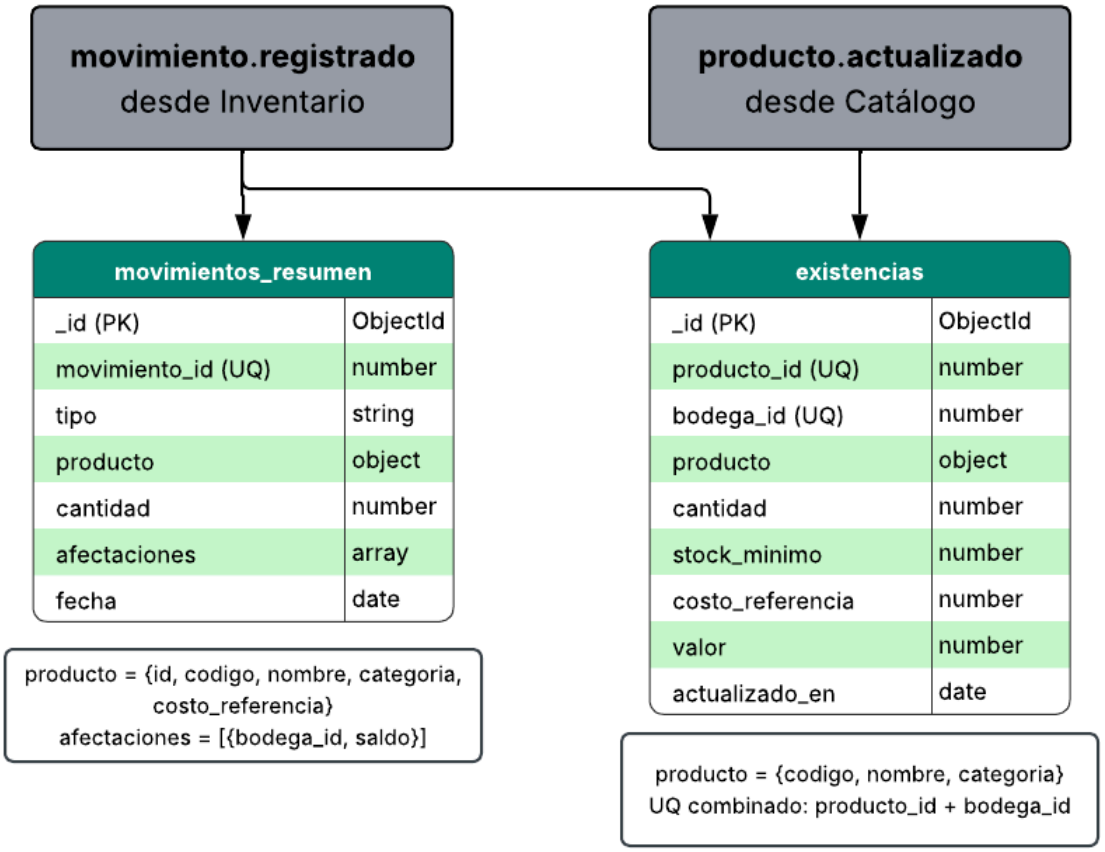


Tabla	Qué guarda	Por qué así	RF
alertas	Los avisos de stock bajo y sugerencias.	Se crean a partir de los eventos de Inventario y Compras. estado pasa de PENDIENTE a ATENDIDA. referencia cambia según el tipo (sea sugerencia u orden), flexibilidad es una de las razones para usar MongoDB.	RF09
suscripciones	Quién recibe qué.	Canal (interfaz, correo o ambos) y bodegas de interés, para que cada jefe de bodega reciba solo lo de su sede.	RF09

- **alertas**: _id, evento_id (UQ), tipo (STOCK_BAJO, TRASLADO_SUGERIDO, ORDEN_SUGERIDA), producto_id, producto_nombre, bodega_id, stock_actual, stock_minimo, estado (PENDIENTE, LEÍDA, ATENDIDA), referencia (Object), envios (Array de Object), creada_en, atendida_en.
- **suscripciones**: _id, usuario_id (UQ), email, canal_interfaz, canal_correo, bodegas (Array), activa.

Referencias: producto_id, bodega_id y usuario_id apuntan a otros servicios.

3.6 Reportes



Rotación e indicadores no se guardan: se calculan por agregación sobre estas dos colecciones. Las colecciones no se relacionan entre sí: sólo comparten valores (producto_id, bodega_id).

Tabla	Qué guarda	Por qué así	RF
movimientos_resumen	Una copia de cada movimiento.	Es la base para calcular la rotación y los indicadores. movimiento_id es único para no contar dos veces el mismo.	RF11
existencias	Saldo y valor por producto y bodega.	valor = cantidad × costo_referencia (costo estándar).	RF11

- **movimientos_resumen:** un documento por movimiento. movimiento_id es único, así no se cuenta dos veces. producto es la "foto" del producto en ese momento, es decir, el histórico. afectaciones guarda el saldo resultante en cada bodega afectada (una, o dos si es un traslado).
- **existencias**: una fila por producto y bodega, con cantidad, mínimo, costo y valor (cantidad × costo).

Referencias: Reportes no lee las bases de otros servicios: todo le llega por los eventos movimiento.registrado y producto.actualizado.

4. Tolerancia a fallos: ¿Qué pasa si cae...?

Si cae	Sigue funcionando	Se afecta	Recuperación
Notificaciones	Todo el registro de inventario	No se envían alertas	Los eventos esperan en su cola y se procesan al volver
Reportes	Todo el registro de inventario	Reportes con datos hasta el último evento	Igual: los eventos esperan en cola
Compras	Movimientos y consulta de stock	No se crean ni reciben órdenes; la orden sugerida se retrasa	El evento queda en cola hasta que vuelva
Catálogo	Consulta de stock, kardex y órdenes ya creadas	No se registran movimientos nuevos ni se crean órdenes (validan el producto)	Mejora posible: caché corta de productos en Inventario
Inventario	Catálogo y consultas de Reportes (última copia)	No hay movimientos ni recepciones	Compras reintenta la recepción
RabbitMQ	Registro de movimientos y consultas	Alertas, sugerencias y reportes se atrasan	Reintentos de publicación; mejora posible: patrón outbox
Gateway	Servicios entre sí	Ningún acceso externo	Varias réplicas en Kubernetes

5. Contratos de API por microservicio

Cada microservicio ofrece sus funciones a los demás componentes mediante una API: un conjunto de **endpoints**, cada uno formado por un método HTTP y una ruta (por ejemplo GET /productos). Un **contrato de API** es el documento que describe, antes de programar, qué endpoints tiene un servicio, qué datos recibe, qué responde, qué errores puede devolver y quién tiene permiso de usarlo. Funciona como un acuerdo entre quienes construyen el servicio y quienes lo consumen (la interfaz web, el canal en línea y los demás microservicios): ambos lados saben qué esperar sin leer el código del otro.

OpenAPI es el estándar: un archivo YAML que describe la API. **Swagger** es el conjunto de herramientas que usan ese archivo; la más conocida, Swagger UI, lo muestra como una página interactiva donde se pueden probar los endpoints. Cada servicio tiene su propio archivo. En la Entrega 3 cada servicio publicará esa documentación en su propia ruta (por ejemplo /api-docs).

Este es un **resumen**: una tabla por servicio con cada endpoint, quién puede usarlo y qué requerimiento cubre. El detalle completo (parámetros, formato de los datos, respuestas y errores) está en los archivos YAML. Los seis contratos fueron validados con una herramienta de validación de OpenAPI 3.0, sin errores.

Servicio	Archivo del contrato	Base (por el Gateway)	Operaciones
API Gateway	gateway-openapi.yaml	/api	14
Catálogo	catalogo-openapi.yaml	/api/catalogo	15
Compras	compras-openapi.yaml	/api/compras	14
Inventario	inventario-openapi.yaml	/api/inventario	18
Notificaciones	notificaciones-openapi.yaml	/api/notificaciones	8
Reportes	reportes-openapi.yaml	/api/reportes	5

En total son 74 operaciones: 68 funciones de negocio y 6 endpoints /health que usa Kubernetes para comprobar que cada servicio está vivo.

Cómo se accede a la API

- Toda petición externa entra por el **API Gateway** (http://localhost:8000 en desarrollo), que enruta según el prefijo: /api/catalogo, /api/compras, etc. (RNF09). Los servicios no se exponen fuera del clúster.
- Entre servicios hay tres llamadas REST internas que no pasan por el Gateway: ① Compras → Catálogo (valida el producto y obtiene su proveedor), ② Inventario →

Catálogo (valida el producto y trae sus datos) y ③ Compras → Inventario (registra la entrada al recibir mercancía). Los endpoints que usan están marcados en las tablas.

- Además de REST, los servicios intercambian eventos por RabbitMQ. No son endpoints; aparecen en la descripción de cada servicio.

Convenciones comunes a los seis contratos

- **Seguridad:** JWT en la cabecera Authorization: Bearer. El Gateway lo valida, revisa el rol y reenvía la identidad del usuario a los servicios (RNF07).
- **Roles:** ADMIN (administra) y OPERADOR (operador de bodega). «Público» significa que no requiere token.
- **Nombres:** rutas en español y en plural, datos JSON en camelCase y fechas en formato ISO 8601.
- **Listas:** paginadas con pagina y limite; la respuesta trae pagina, limite, total y datos.
- **Errores:** un formato único (codigo, mensaje, detalles) y los mismos códigos HTTP en todos los servicios: 400 datos inválidos, 401 sin sesión, 403 sin permiso, 404 no existe, 409 conflicto con el estado actual y 503 un servicio del que depende no responde.
- **Bajas lógicas:** DELETE no borra el registro, lo marca como inactivo, para conservar el historial.
- **Idempotencia:** POST /movimientos acepta la cabecera Idempotency-Key, para que un reintento no duplique un movimiento.

5.1 Gateway (autenticación y usuarios)

Base: /api. Atiende directamente la autenticación, los usuarios y los roles (RF07, RF08) y enruta (proxy) todas las rutas /api/{servicio}/** hacia los microservicios. No publica ni consume eventos.

Método	Ruta	Rol	Qué hace	RF
GET	/health	Público	Estado del Gateway (probes de Kubernetes)	RNF04
POST	/auth/login	Público	Inicia sesión y devuelve el JWT	RF08
POST	/auth/logout	ADMIN, OPERADOR	Cierra la sesión: revoca el token actual	RF08
GET	/auth/perfil	ADMIN, OPERADOR	Datos y rol del usuario autenticado	RF08
POST	/auth/recuperar	Público	Solicita recuperar la contraseña; envía un enlace por correo	RF08
POST	/auth/restablecer	Público	Define la nueva contraseña con el token recibido por correo	RF08

POST	/auth/cambiar-password	ADMIN, OPERADOR	Cambia la contraseña del usuario autenticado	RF08
GET	/usuarios	ADMIN	Lista y busca usuarios	RF07
POST	/usuarios	ADMIN	Registra un usuario	RF07
GET	/usuarios/{usuarioId}	ADMIN	Consulta un usuario	RF07
PUT	/usuarios/{usuarioId}	ADMIN	Actualiza un usuario o lo reactiva	RF07
DELETE	/usuarios/{usuarioId}	ADMIN	Desactiva un usuario (baja lógica)	RF07
PUT	/usuarios/{usuarioId}/rol	ADMIN	Asigna un rol a un usuario	RF07
GET	/roles	ADMIN	Lista los roles disponibles	RF07

5.2 Catálogo

Base: /api/catalogo. Productos, categorías y asociación producto–proveedor. Publica el evento catalogo.producto.actualizado hacia Reportes. Sus endpoints de consulta de producto los usan también Compras (①) e Inventario (②).

Método	Ruta	Rol	Qué hace	RF
GET	/health	Público	Estado del servicio	RNF04
GET	/productos	ADMIN, OPERADOR	Busca y filtra productos por nombre, código, categoría o proveedor	RF01, RF12
POST	/productos	ADMIN	Crea un producto	RF01
GET	/productos/{productoId}	ADMIN, OPERADOR	Consulta un producto. También lo usan Compras (①) e Inventario (②)	RF01
PUT	/productos/{productoId}	ADMIN	Actualiza un producto; si cambia el nombre, la categoría, el costo o el estado, avisa a Reportes por evento	RF01
DELETE	/productos/{productoId}	ADMIN	Da de baja un producto (baja lógica)	RF01
GET	/productos/{productoId} /proveedores	ADMIN, OPERADOR	Lista los proveedores de un producto. También lo usa Compras (①)	RF01, RF03
POST	/productos/{productoId} /proveedores	ADMIN	Asocia un proveedor a un producto	RF03
PUT	/productos/{productoId} /proveedores/{proveedorId}	ADMIN	Marca o desmarca el proveedor principal	RF03

DELETE	/productos/{productoId} /proveedores/{proveedorId}	ADMIN	Quita un proveedor de un producto	RF03
GET	/categorias	ADMIN, OPERADOR	Lista categorías y subcategorías (plana o en árbol)	RF02
POST	/categorias	ADMIN	Crea una categoría o subcategoría	RF02
GET	/categorias/{categoriaId}	ADMIN, OPERADOR	Consulta una categoría	RF02
PUT	/categorias/{categoriaId}	ADMIN	Actualiza o reactiva una categoría	RF02
DELETE	/categorias/{categoriaId}	ADMIN	Da de baja una categoría, si no tiene productos ni subcategorías activos	RF02

5.3 Compras

Base: /api/compras. Proveedores, órdenes de compra y recepción de mercancía. Consume el evento inventario.reabastecimiento.requerido (crea la orden sugerida en borrador) y publica compras.orden.sugerida. Llama a Catálogo (①) y a Inventario (③).

Método	Ruta	Rol	Qué hace	RF
GET	/health	Público	Estado del servicio	RNF04
GET	/proveedores	ADMIN, OPERADOR	Lista y busca proveedores	RF03
POST	/proveedores	ADMIN	Registra un proveedor	RF03
GET	/proveedores/{proveedorId}	ADMIN, OPERADOR	Consulta un proveedor	RF03
PUT	/proveedores/{proveedorId}	ADMIN	Actualiza o reactiva un proveedor	RF03
DELETE	/proveedores/{proveedorId}	ADMIN	Da de baja un proveedor, si no tiene órdenes abiertas	RF03
GET	/ordenes-compra	ADMIN, OPERADOR	Lista órdenes con filtros (estado, origen, proveedor, bodega, fechas)	RF06
POST	/ordenes-compra	ADMIN	Crea una orden en borrador; valida los productos en Catálogo (①)	RF06
GET	/ordenes-compra/{ordenId}	ADMIN, OPERADOR	Consulta una orden con sus líneas	RF06

PUT	/ordenes-compra/{ordenId}	ADMIN	Edita una orden que sigue en borrador	RF06
POST	/ordenes-compra/{ordenId}/enviar	ADMIN	Aprueba y envía la orden al proveedor; es el paso de aprobación de las órdenes sugeridas	RF06, RF16
POST	/ordenes-compra/{ordenId}/cancelar	ADMIN	Cancela una orden que aún no tiene recepciones	RF06
GET	/ordenes-compra/{ordenId}/recepciones	ADMIN, OPERADOR	Lista las recepciones de una orden	RF06
POST	/ordenes-compra/{ordenId}/recepciones	ADMIN, OPERADOR	Recibe mercancía, total o parcial: registra la entrada en Inventario (③) y actualiza el estado de la orden	RF06

5.4 Inventario

Base: /api/inventario. Bodegas, stock por bodega, movimientos y kardex, devoluciones y sugerencias de reabastecimiento. Publica inventario.stock.bajo, inventario.movimiento.registrado e inventario.reabastecimiento.requerido; no consume eventos. Llama a Catálogo (②) y lo llama Compras (③).

Método	Ruta	Rol	Qué hace	RF
GET	/health	Público	Estado del servicio	RNF04
GET	/bodegas	ADMIN, OPERADOR	Lista las bodegas	RF05
POST	/bodegas	ADMIN	Crea una bodega	RF05
GET	/bodegas/{bodegaId}	ADMIN, OPERADOR	Consulta una bodega	RF05
PUT	/bodegas/{bodegaId}	ADMIN	Actualiza una bodega	RF05
DELETE	/bodegas/{bodegaId}	ADMIN	Da de baja una bodega, si ya no tiene stock	RF05
GET	/bodegas/{bodegaId}/stock	ADMIN, OPERADOR	Stock de todos los productos de una bodega; puede mostrar solo los que están bajo el mínimo	RF05
PUT	/bodegas/{bodegaId}/stock/{productoId}/minimo	ADMIN	Configura el stock mínimo de un producto en una bodega	RF09

GET	/stock/{productoId}	ADMIN, OPERADOR	Stock de un producto en cada bodega y en total; es la consulta que usará el canal en línea	RF05, RF15
GET	/movimientos	ADMIN, OPERADOR	Lista movimientos con filtros (producto, bodega, tipo, fechas)	RF04, RF10
POST	/movimientos	ADMIN, OPERADOR	Registra una entrada, salida, ajuste o traslado y actualiza el saldo. También lo usa Compras (③)	RF04
GET	/movimientos/{movimientoId}	ADMIN, OPERADOR	Consulta un movimiento	RF10
GET	/kardex	ADMIN, OPERADOR	Historial de un producto con entradas, salidas y saldo	RF10
GET	/devoluciones	ADMIN, OPERADOR	Lista devoluciones	RF13
POST	/devoluciones	ADMIN, OPERADOR	Registra una devolución de cliente (entra stock) o a proveedor (sale stock)	RF13
GET	/sugerencias-reabastecimiento	ADMIN, OPERADOR	Lista las sugerencias de traslado o compra	RF16
POST	/sugerencias-reabastecimiento/{sugerenciaId}/aceptar	ADMIN	Acepta una sugerencia de traslado y registra el traslado	RF16
POST	/sugerencias-reabastecimiento/{sugerenciaId}/rechazar	ADMIN	Rechaza una sugerencia	RF16

5.5 Notificaciones

Base: /api/notificaciones. Alertas en la interfaz y por correo. Consume inventario.stock.bajo y compras.orden.sugerida; no publica eventos ni llama a otros servicios.

Método	Ruta	Rol	Qué hace	RF
GET	/health	Público	Estado del servicio	RNF04
GET	/alertas	ADMIN, OPERADOR	Lista alertas con filtros (estado, tipo, bodega)	RF09, RF16
GET	/alertas/resumen	ADMIN, OPERADOR	Cantidad de alertas pendientes (indicador en la interfaz)	RF09
GET	/alertas/{alertaId}	ADMIN, OPERADOR	Consulta una alerta y el estado de sus correos	RF09

POST	/alertas/{alertaId}/leer	ADMIN, OPERADOR	Marca una alerta como leída	RF09
POST	/alertas/{alertaId}/atender	ADMIN, OPERADOR	Marca una alerta como atendida	RF09
GET	/preferencias	ADMIN, OPERADOR	Consulta las preferencias de notificación del usuario	RF09
PUT	/preferencias	ADMIN, OPERADOR	Define los canales (interfaz y correo) y las bodegas de interés	RF09

5.6 Reportes

Base: /api/reportes. Modelo de lectura alimentado por eventos: consume inventario.movimiento.registrado y catalogo.producto.actualizado; no llama a otros servicios. Todos sus endpoints son solo para ADMIN, porque exponen costos y valorización.

Método	Ruta	Rol	Qué hace	RF
GET	/health	Público	Estado del servicio	RNF04
GET	/reportes/existencias	ADMIN	Existencias por producto y bodega; con bajoMinimo lista los quiebres de stock	RF11
GET	/reportes/valorizacion	ADMIN	Valor del inventario agrupado por bodega o por categoría	RF11
GET	/reportes/rotacion	ADMIN	Productos de mayor o menor rotación en un periodo	RF11
GET	/indicadores	ADMIN	Panel: valor total, quiebres de stock y rotación promedio	RF14

5.7 Contratos de eventos (RabbitMQ)

Todos los eventos llevan un id único y la fecha de emisión; el consumidor usa ese id para no procesar dos veces el mismo mensaje.

Evento	Publica	Consume	Contenido
inventario.movimiento.registrado	Inventario	Reportes	eventoId, fecha, movimientoId, tipo, cantidad, producto (id, codigo, nombre, categoria, costoReferencia), afectaciones (por bodega: bodegaId, saldo, stockMinimo; dos si es traslado)
inventario.stock.bajo	Inventario	Notificaciones	eventoId, fecha, productoId, productoNombre, bodegaId, stockActual, stockMinimo y, si hay traslado sugerido, sugerencia (id, bodegaOrigenId, cantidad)
inventario.reabastecimiento.sugerido	Inventario	Compras	eventoId, fecha, sugerenciaId, productoId, bodegaDestinoId, cantidad
compras.orden.sugerida	Compras	Notificaciones	eventoId, fecha, ordenId, numero, productoId, productoNombre, proveedorId, bodegaId, cantidad
catalogo.producto.actualizado	Catálogo	Reportes	eventoId, fecha, productoId, codigo, nombre, categoria (id, nombre), costoReferencia, activo

Anexo A. Bitácora de uso de IA - Entrega 2

Proyecto	Sistema de Gestión de Inventarios ~ FerreAndina S.A.S.
Equipo / Integrantes	Felipe Giraldo, Jhoan Sebastián, Steven Lopez
Entrega N.°	2
Periodo cubierto	30/09/2026 – 26/10/2026

Fecha	Integrante	Herramienta	Tarea apoyada	Prompt (resumen)	Interv. humana	Resultado / aprendizaje
17/10/2026	Sebastián Martínez	Claude	Diagrama de arquitectura de microservicios y explicación de cambios.	Validar y corregir el diagrama de arquitectura de microservicios realizado en LucidChart, explicando correcta y claramente los cambios.	Media	Identificó la manera correcta del manejo de eventos con RabbitMQ y corrigió, al igual que algunas relaciones directas entre microservicios con su explicación lógica y funcional.
18/10/2026	Sebastián Martínez	Claude	Apoyo en la creación del modelo de datos de cada microservicio tomando en cuenta los requerimientos tanto funcionales como no funcionales.	Apoyar la creación e implementación lógica de los modelos de datos de cada microservicio teniendo en cuenta los RF y RNF del proyecto.	Alta	Creación de modelos de datos para cada microservicio. Resultado que fue minuciosamente revisado, encontrando tablas innecesarias que fueron eliminadas, además de, llaves foráneas y referencias lógicas fuera de lugar corregidas exitosamente.
18/10/2026	Steven Lopez Parra	Claude	Funcionamiento y explicación de manejo de eventos con RabbitMQ	Explicar detalladamente cómo se manejan los eventos asíncronos con la ayuda del broker RabbitMQ.	Baja	Manejo de eventos asíncronos con la ayuda de RabbitMQ, así como los eventos principales usados en el proyecto.
19/10/2026	Felipe Giraldo	Claude	Revisión del modelo de datos.	Revisar las tablas de los seis servicios contra el enunciado y los RF/RNF de la Entrega 1 y	Media	Encontró la llave primaria de stock incompleta (solo producto_id), datos de Reportes sin origen y campos repetidos en devoluciones. Se decidió

Fecha	Integrante	Herramienta	Tarea apoyada	Prompt (resumen)	Interv. humana	Resultado / aprendizaje
				proponer correcciones.		fusionar devoluciones en movimientos y valorizar a costo estándar, y verificó cada cambio contra los RF antes de pasarlo al Lucidchart.
20/10/2026	Felipe Giraldo	Claude	Contratos de API y justificación de las bases de datos.	Redactar los contratos de API de cada servicio y explicar por qué Reportes y Notificaciones tienen base propia en MongoDB.	Media	Revisamos cada endpoint contra los RF y los roles del Gateway. La justificación de MongoDB la ajustamos tras discutirla: MySQL también serviría, y la ventaja real es la flexibilidad del esquema.
23/10/2026	Steven López Parra	Claude	Corrección y complementación en los modelos de bases de datos.	Analiza los diagramas de base de datos frente a los requerimientos funcionales y aporta correcciones o complementaciones. Explica el por qué de cada decisión.	Alta	Se valida el esquema inicialmente corregido, confrontando cada diagrama con sus RF y RNF asociados. Refuerza la utilización de RabbitMQ para la gestión de eventos garantizando idempotencia (no duplicados).

Reflexión final de la fase

En esta fase delegamos en la IA las tareas de borrador y de contraste: validar el diagrama de arquitectura de microservicios, proponer y revisar el modelo de datos de cada servicio contra los requerimientos de la Entrega 1, y redactar los contratos de API en formato OpenAPI, junto con el resumen de contratos de este documento. Lo hicimos para ganar velocidad y mantener las mismas convenciones en los seis servicios, pero ningún resultado se aceptó sin una verificación manual contra el enunciado y la Entrega 1, y los contratos se validaron además con una herramienta de validación de OpenAPI 3.0.

Las decisiones de diseño las tomamos nosotros: fusionar las devoluciones en los movimientos, valorizar a costo estándar, mantener base propia en Reportes y Notificaciones (simplificando sus colecciones) y conservar el evento de actualización de productos. La revisión encontró errores que habíamos pasado por alto, como la llave primaria de stock, que solo permitía tener cada producto en una bodega, y campos de Reportes que no tenían de dónde llegar (la rotación necesitaba un stock promedio que el modelo no guardaba). También detectamos sobrediseño, con colecciones que sobraban, y posibles duplicados por reintentos o eventos repetidos, que resolvimos con índices únicos y una clave de idempotencia. Con respecto a la primera entrega, se afirma la utilización de RabbitMQ para la gestión de eventos asincrónicos definidos. Con REST, cada uno de estos procesos debe esperar necesariamente la respuesta del otro para poder continuar, es decir, si alguno de ellos falla, el registro de movimientos queda en pausa, situación que resuelve RabbitMQ, se emite el evento el cual se almacena en una cola, el funcionamiento continúa y se obtiene respuesta en el momento en que el consumidor esté disponible. Además, la IA citó una vez un requerimiento que no existe y dejó algunos contratos desalineados con el modelo de datos. Aprendimos que la IA acelera el borrador, pero la coherencia entre diagrama, modelo, contratos y requerimientos solo se logra revisando cada pieza contra las demás.

Referencias

[1] MongoDB Inc. *Data Modeling*. https://www.mongodb.com/docs/manual/data-modeling/ (documentos embebidos, Object y Array)

[2] OpenAPI Initiative. *OpenAPI Specification v3.0.3*. https://spec.openapis.org/oas/v3.0.3 (contratos de API)

[3] Newman, S. *Building Microservices* (2.ª ed.). O'Reilly, 2021. (diseño de servicios y comunicación entre ellos)

[4] SmartBear. *Swagger Documentation*. https://swagger.io/docs/ (Swagger UI)

[5] Broadcom. *RabbitMQ Tutorials*. https://www.rabbitmq.com/tutorials (exchanges, colas y eventos)

[6] Claude. https://claude.ai

Repositorio de Github del Proyecto

https://github.com/SebasMartinez21/Microservicios_FerreAndina